SpinalHDL Audio Oscillator Implementation Specification

Table of Contents

1. spinalHDL Hierarchy

The spinalHDL implementation shall use the following hierarchy for folders, subfolders and files.

```
Simple_Oscillator_SpinalHDL/
├── build.sbt                   # SBT build configuration (dependencies, Scala version)
├── project/                    # SBT plumbing
│   └── build.properties        # Defines the SBT version
├── src/
│   ├── main/
│   │   └── scala/
│   │       └── synth/          # Root package for the project
│   │           ├── Synth.scala             # Top-level System Integration (per Section 2)
│   │           ├── TimingGenerator.scala   # Tick generation logic (per Section 3)
│   │           ├── uart/                   # Control Path logic
│   │           │   ├── UartRx.scala              # UART Receiver
│   │           │   ├── UartProtocolDecoder.scala # Protocol Parser
│   │           │   └── RegisterBank.scala        # Parameter Storage
│   │           ├── oscillator/             # Core Oscillator logic (per Section 4)
│   │           │   ├── Oscillator.scala    # Main Oscillator module
│   │           │   ├── Accumulator.scala   # Phase logic
│   │           │   ├── Noise.scala         # LFSR logic
│   │           │   ├── Generators.scala    # Waveform logic (Saw, Tri, etc.)
│   │           │   └── Mux.scala           # Waveform selection
│   │           ├── output/                 # Audio output pipeline
│   │           │   ├── Decimator.scala     # 10x downsampling (per Section 5)
│   │           │   └── I2STransmitter.scala # I2S protocol engine (per Section 6)
│   └── test/
│       └── scala/
│           └── synth/          # SpinalSim testbenches
│               ├── SynthSim.scala          # Full system simulation
│               ├── TimingSim.scala         # Verifying tick precision
│               ├── oscillator/
│               │   └── WaveformSim.scala   # Verifying Generator math
│               └── output/
│                   └── I2STransmitterSim.scala # Verifying I2S timing/protocol
├── rtl/                        # Output folder for generated Verilog/VHDL files
├── doc/                        # Architecture diagrams and design assets
├── README.md                   # Project overview (Context File)
└── Implementation_specs.md     # Technical specification (Context File)
```

2. Synth (System Top) Module

Purpose

The Synth module is the hardware entry point and system integration entity.

The module shall:

- Instantiate UART control (Rx, Decoder, Registers)
- Instantiate the Synthesis Engine (Timing, Oscillator, Output)
- connect subsystem interfaces
- expose the external hardware interface
- contain no DSP or protocol implementation details

Clocking and Reset

The system operates within a single clock domain managed at the top level:

- **Clock**: 24 MHz external input.
- **Reset**: Asynchronous, Active-High.

IO Bundle

```
val io = new Bundle {
    val clk        = in Bool()
    val reset      = in Bool()

    val freqWord   = in UInt(24 bits)
    val waveSelect = in UInt(3 bits)
    val pwmWidth   = in UInt(8 bits)

    val i2s_bclk   = out Bool()
    val i2s_lrclk  = out Bool()
    val i2s_sdata  = out Bool()
}
```

3. TimingGenerator

IO Bundle

```
val io = new Bundle {

    val phaseTick  = out Bool()
    val sampleTick = out Bool()
}
```

Internal Structure

Signal	Width	Purpose
`phaseCounter`	6 bit	modulo-50 counter
`sampleCounter`	9 bit	modulo-500 counter

Both counters:

- operate directly from the 24 MHz master clock
- run independently
- are free-running

Tick Behaviour

Both tick outputs:

- are registered
- synchronous
- one clock cycle wide
- default to `False`

4. Uart and Register Bank

This sub-system handles external control, parsing incoming serial data and storing configuration parameters.

4.1 UartRx

Purpose: Converts the serial UART bitstream into parallel 8-bit bytes. It operates at 115,200 baud using a 208-tick bit period and includes start-bit verification.

IO Bundle

```
val io = new Bundle {
    val rx        = in Bool()
    val data      = out Bits(8 bits)
    val dataValid = out Bool()
}
```

4.2 UartProtocolDecoder

Purpose: Frames individual bytes into 3-byte command packets: `[Address/Command]`, `[Data]`, and `[Reserved]`. It asserts `writeEnable` only when a full frame is valid.

IO Bundle

```
val io = new Bundle {
    val rxData      = in Bits(8 bits)
    val rxDataValid = in Bool()
    val writeEnable  = out Bool()
    val writeAddress = out UInt(8 bits)
    val writeData    = out Bits(8 bits)
}
```

4.3 RegisterBank

Purpose: Stores the current state of the synthesizer parameters. It implements an atomic update for the 24-bit frequency word, ensuring all three bytes are applied simultaneously to the DDS engine upon writing to the high-byte address.

IO Bundle

```
val io = new Bundle {
    val writeEnable  = in Bool()
    val writeAddress = in UInt(8 bits)
    val writeData    = in Bits(8 bits)

    val oscFrequency  = out UInt(24 bits)
    val oscWaveform   = out UInt(8 bits)
    val oscPulseWidth = out UInt(8 bits)
    val oscVolume     = out UInt(8 bits)
}
```

5. Oscillator

The Oscillator is a module that connects the four submodules, as shown in the following internal submodule structure. It serves as an abstraction layer above the generation of the oscillating audio signals.

Purpose

Core DDS synthesis engine.

Responsibilities:

- phase accumulation
- waveform generation
- noise generation
- mux between waveforms and noise
- oversampled sample generation

IO Bundle

```
val io = new Bundle {
    val phaseTick = in Bool()
    val freqWord  = in UInt(24 bits)
```

```
    val waveSelect = in UInt(3 bits)
    val pwmWidth  = in UInt(8 bits)
    val sample    = out SInt(16 bits)
}
```

Internal structure of submodules:

```
Oscillator
 ├── Accumulator
 ├── Noise
 ├── Generators
 └── Mux
```

5.1 Oscillator Submodules

Accumulator

Responsible for:

- phase register
- phase accumulation
- frequency addition

```
val io = new Bundle {
    val phaseTick = in Bool()
    val freqWord  = in UInt(24 bits)
    val phase     = out UInt(24 bits)
}
```

Noise

Responsible for:

- LFSR register
- feedback logic
- shift/update logic
- noise sample generation

```
val io = new Bundle {
    val phaseTick = in Bool()
    val sample    = out SInt(16 bits)
}
```

Generators

Responsible for:

- saw generation
- square generation
- PWM generation
- triangle generation

```
val io = new Bundle {
    val phase      = in UInt(24 bits)
    val pwmWidth   = in UInt(8 bits)
    val sawWave    = out SInt(16 bits)
    val squareWave = out SInt(16 bits)
    val pwmWave    = out SInt(16 bits)
    val triWave    = out SInt(16 bits)
}
```

The module shall contain only combinational logic.

Mux

Responsible for:

- waveform or noise selection
- final waveform routing

```
val io = new Bundle {

    val waveSelect = in UInt(3 bits)
    val sawWave    = in SInt(16 bits)
    val squareWave = in SInt(16 bits)
    val pwmWave    = in SInt(16 bits)
    val triWave    = in SInt(16 bits)
    val noiseWave  = in SInt(16 bits)
    val sample     = out SInt(16 bits)
}
```

5.2 Oscillator signal flow

```
Accumulator ── phase ──┐
                       │
                       ↓
                  Generators
                       │
                       ├── sawWave
                       ├── squareWave
                       ├── pwmWave
                       └── triangleWave

Noise ── noiseSample ──┘
                       ↓
                     Mux
                       ↓
                   sample
```

6. Decimator

IO Bundle

```
val io = new Bundle {
    val phaseTick  = in Bool()
    val sampleTick = in Bool()
    val sampleIn   = in SInt(16 bits)
    val sampleOut  = out SInt(16 bits)
    val valid      = out Bool()
}
```

Responsible for converting the 480 kHz oversampled waveform stream into 48 kHz audio samples. The decimator captures every 10th oscillator sample.

sampleOut is:

- registered
- stable
- updated only at 48 kHz

valid communicates that a new 48 kHz sample is available now.

7. I2STransmitter

IO Bundle

```
val io = new Bundle {
    val sampleIn  = in SInt(16 bits)
    val valid     = in Bool()
    val bclk      = out Bool()
    val lrclk     = out Bool()
    val sdata     = out Bool()
}
```

Self-contained serializer and protocol engine.

Responsibilities:

- BCLK generation
- LRCLK generation
- shift register control
- stereo frame timing
- serial audio output

The module operates directly from the 24 MHz master clock.

The serializer uses the scheduled timing subpattern:

```
16,16,15,16,16,15,16,15
```

to generate the required average I²S bit timing.

7.1 Gated Startup and Idle State

The transmitter employs a gated startup mechanism to ensure that the I2S clock and data lines only toggle when valid audio data is present.

Idle Behavior: The transmitter starts in an inactive state after reset:

- `bclk` and `sdata` are held `Low`.
- `lrclk` is held `High` (the standard idle state for I2S).

Activation: The state machine transitions to `active` upon the first assertion of `io.valid`. On this cycle:

- Internal counters (`bitCounter`, `cycleCounter`, `patternIndex`) are reset to `0`.
- The input sample is latched into the `sampleBuffer`.
- The `shiftRegister` is loaded, and transmission of the Left channel begins immediately.

Once active, the transmitter remains in the active state to maintain a continuous bit clock, even if subsequent `valid` pulses are delayed, though it will re-synchronize its frame boundaries to the `valid` signal to prevent drift.